Autoresearch Loops: Causal Study of the Instruction File

Christian Block
Research project with Prof. Ramesh

September 3, 2026

1 Introduction

In an autoresearch loop, the experimentation itself is performed by a language model. The model is given a goal, a body of code it may edit, and an executor that runs a trial and returns a scalar result. From that point it proceeds autonomously: it proposes a change, executes it, reads the outcome, decides what to attempt next, and repeats until the budget is exhausted. The current meaning of the term was fixed by Karpathy's `autoresearch` [5], which appeared in March 2026 and has since been forked several thousand times.

The architecture is a contract between three files, each with a single owner. `prepare.py` prepares the data and defines the score; it is modified by no one, so that every experiment is scored identically. `train.py` contains the model and the training loop and is edited only by the agent, while `program.md` contains the instructions and is edited only by the human. Each candidate is allocated five minutes of wall-clock time and yields a single number, validation bits per byte, so that trials are equally costly and directly comparable. Progress is enforced through version control: a change is committed and trained, the commit is retained if the score improved, and otherwise reverted by `git reset` [2]. The repository only ever advances.

What the system does not contain is orchestration code. There is neither a scheduler nor a driver program: the nine-step experiment cycle exists only as English prose within `program.md`, and the model itself is the execution layer. `program.md` is therefore not configuration supplied to a controller; it is the controller.

Such loops demonstrably function, but the reasons for their behaviour remain unknown. The AI Scientist [6] and Coscientist [3] are evaluated end to end, on whether the pipeline produced an acceptable artefact, and benchmarks such as MLAgentBench [4] report success rates across task suites. Industrial reports take the same form: an autoresearch run against a production template engine reduced parse-and-render time by 53% over approximately one hundred and twenty experiments, a gain that its own author considered probably overfitted and that was never merged [1]. Such evidence establishes that a system can work; it does not establish which part of the instruction file was responsible.

The obstacle to such an attribution is confounding, since an instruction file settles several matters simultaneously: what counts as success, how widely the agent may search, when it must stop, how it must report, and where effort should be concentrated. When two differently worded files are compared, every sentence differs, and no individual effect is identified. Work on prompt sensitivity sharpens the problem further: formatting choices that preserve meaning can shift accuracy by as much as 76 points [7], whereas models sometimes perform equally well on instructions that are irrelevant or actively misleading [8]. Wording evidently matters, but inspection of the wording does not reveal how.

Establishing how is an empirical problem. This project treats the instruction file not as prose but as a

configuration vector. Five components of `program.md` are each written at two levels, and all $2^5 = 32$ combinations are generated and executed against a real executor, which separates the effect of each component instead of assuming it. The contribution is a method rather than an architecture: the aim is not to propose a superior autoresearch loop, but to establish what an existing one is responding to.

The scope is deliberately narrow: no comparison is drawn between the loop and a human researcher, and neither a human baseline nor an external standard of good research practice is invoked. Every comparison is made between two configurations of the same system.

RQ1 Which components of `program.md` causally determine which aspects of the loop's research *process*, and how large is each effect?

RQ2 Do the components act independently of one another, or does the effect of one depend on the level of another?

The remainder of the paper is organised as follows. Section 2 sets out the loop and its configuration space, Section 3 presents the design, the results and their interpretation, and Section 4 concludes.

2 Background

The loop under study. The loop examined here follows the protocol described above, subject to one narrowing: the search space is a hyperparameter space rather than arbitrary code, so that the instruction file remains the only free variable. The file is supplied once as the system message and is not modified thereafter. The model then alternates between two moves, each expressed as a single JSON object. A `propose` carries a hypothesis, a model family and a set of hyperparameter values, whereas an `interpret` carries a reading of the most recent measurement together with a decision to continue or to stop.

Execution is performed by the harness, which validates each proposal against a fixed whitelist and fits the estimator using scikit-learn. The model never computes a number; it selects what is computed and reads the result. Every value in a transcript can be traced to one of two origins: it was produced by the executor, in which case it is a fact about the task, or by the model, in which case it is a fact about the loop's behaviour. Figure 1 shows the resulting control flow.

The instruction file as a configuration. Every instruction file is generated from a single template. The fixed part specifies the research question, the executor interface and the JSON response format, and is byte-identical across all thirty-two files. The variable part consists of five paragraphs, each written at one of two levels; these five slots constitute the independent variables, and nothing else in the file varies. A file is described completely by

$$c = (c_1, \ldots, c_5), \qquad c_i \in \{0, 1\},$$

where the slots are taken in the order metric, breadth, stopping rule, output format and emphasis, yielding $2^5 = 32$ files.

Table 1 lists the levels, which were not invented for this study: a production `program.md` hardcodes the baseline to be exceeded, specifies how failures are to be handled, fixes a reporting discipline and imposes a parsimony rule [2]; the five components below are drawn from those categories.

The ten paragraphs are written by a language model from fixed meta-prompts, then frozen and reused verbatim. This procedure is a control: had the paragraphs been composed by the author, the wording of every treatment would have constituted a free parameter confounded with the author's own stylistic habits, and no measured effect could have been separated from it. The generating model is distinct from the agent. Each paragraph is verified to entail its intended level and to make no reference to any other slot, and the two levels of a given slot are matched to within ten per cent in token count.

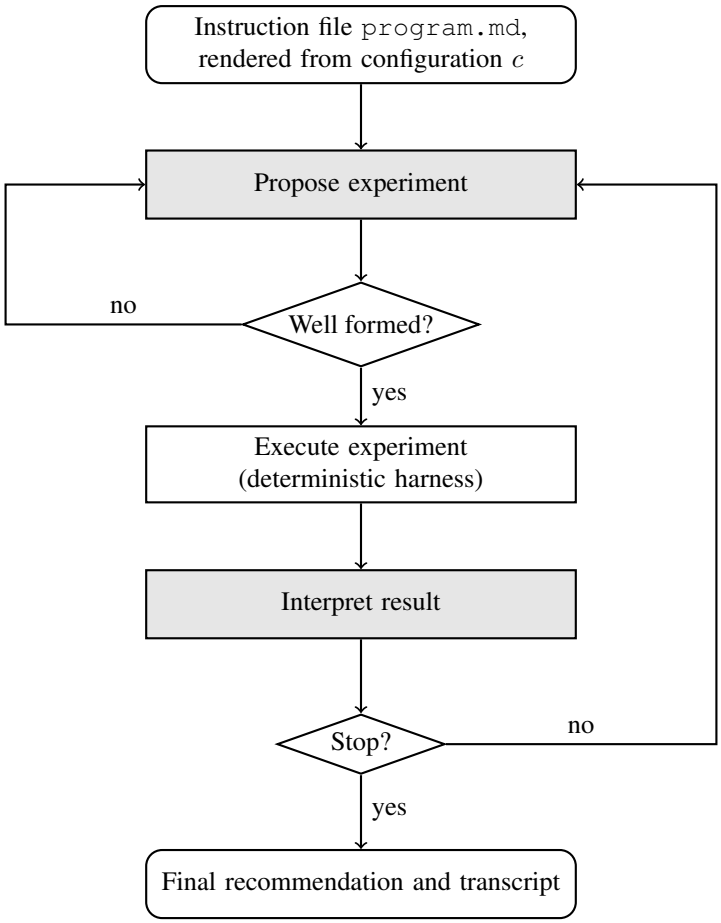

Figure 1: Control flow of one run. Shaded boxes are language-model calls. The unshaded box is deterministic, so every number the loop reasons about was measured and not generated. A malformed or rejected proposal goes back to the proposal step without spending an experiment. The loop halts on the model's decision or on the harness budget, whichever comes first.

Configuration change as intervention. The configurations are constructed rather than observed: for each of the thirty-two values of c the corresponding file is generated and executed. In observational prompt data the components would be correlated, since an author who writes a precise success criterion is also likely to write a careful stopping rule. Construction breaks that correlation. Across the thirty-two files, each component is at level 1 in exactly sixteen, and within each half the remaining four components remain balanced. The difference in mean outcome between runs with $c_i = 1$ and runs with $c_i = 0$ therefore cannot be produced by any other component. Confounding is eliminated by the design itself, with no statistical adjustment.

Executing the complete space yields more than the five main effects. Given all thirty-two files, every pairwise interaction is estimated separately from every main effect, so that the independence of the components becomes a question answerable from the data. A design covering only part of the space must instead assume an answer, since the runs required to distinguish a main effect from an interaction are precisely those it omits. Thirty-two cells is the cost of avoiding that assumption, and at this scale the cost is acceptable.

3 Numerical Evaluation

Tasks and executor. Every configuration is executed against two canonical tabular classification datasets, selected to differ in size, dimensionality and number of classes: Breast Cancer Wisconsin

Table 1: The five varied components of `program.md`. A configuration is one choice of level per row.

Code	Component	Level 0	Level 1
M	Evaluation criterion	Success left undefined; the loop judges quality as it sees fit.	Success defined as five-fold cross-validated accuracy, with the across-fold standard deviation used only to break ties.
B	Search breadth	Commit to one model family for the session.	Compare both families within a session and vary several hyperparameters at once.
S	Stopping rule	Fixed budget of exactly three experiments.	Adaptive budget of up to eight, halted when an experiment fails to improve by more than 0.002.
O	Reporting style	Terse: the action taken and the numbers involved.	Full sentences stating hypothesis, expectation and interpretation.
E	Search emphasis	Explore first: several different configurations before refining any.	Exploit first: refine a promising configuration immediately.

(569×30, two classes) and Wine (178×13, three classes). Each employs a single stratified $70/30$ partition, fixed with the same seed throughout. The loop may select either `RandomForestClassifier` or `GradientBoostingClassifier` under a fixed hyperparameter whitelist. Whatever a configuration's evaluation paragraph emphasises, the executor returns an identical record in every case: five-fold cross-validated accuracy, its across-fold standard deviation, and held-out validation accuracy. The instruction file alters what the loop attends to; it does not alter what is measured.

Baseline. A single reference quantity is computed per dataset before any run, by a procedure involving no language model. The quantity $a_0(d)$ is the accuracy of a default-hyperparameter fit, taken as the better of the two model families, and every run is measured against it. No exhaustive grid search is required, which keeps the reference inexpensive and excludes an estimated quantity from the measurement.

One limitation of the task suite follows from this procedure. The 54-instance validation split of the second dataset is classified perfectly by default hyperparameters, so that a_0 equals 1.0 and no run can improve upon it. On that task the gain is measured on the cross-validated scale alone. This is a property of the task, not of the metric.

Control of non-treatment variation. These are held fixed and so cannot explain any measured effect: the agent model, pinned to the dated snapshot `gpt-4o-mini-2024-07-18` instead of a moving alias; the decoding temperature, one value for every run; the partition and its seed; the executor and its whitelist; the fixed part of the template; the response format; and the harness safety caps. Runs are separate conversations, so nothing carries over between them.

Sampling noise is treated differently: it is controlled rather than eliminated. Every run passes an explicit sampling seed to the model interface, so that any run can be regenerated exactly, and every cell draws from the same seed list, so that the draws cannot favour one configuration. Greedy decoding at temperature zero would be inappropriate here, since run-to-run variance is the error term against which every coefficient below is tested; a deterministic loop would drive that variance to zero and with it every standard error. The temperature is held constant and strictly above zero, which makes it a controlled constant. When a run fails to produce a valid experiment it is repeated with a fresh seed until the cell contains exactly R completed runs, so that balance is preserved by construction.

Design and budget. The design is the complete 2^5 factorial crossed with the task suite and replicated R times, giving $N = 2^5 \cdot D \cdot R$ runs. With $D = 2$ and $R = 20$, this yields $N = 1280$. The replicate count was not chosen for convenience: it fixes the smallest detectable level switch at $0.22\,\sigma$ per dataset, where σ denotes the run-to-run standard deviation of whichever metric is being fitted.

Metrics. The metrics are derived directly from the structured transcript. No evaluation framework, scoring library or judge model is involved, since every metric is a difference in accuracy, a proportion of the proposals, or a count of executor calls.

Measurement window. The stopping rule determines how many experiments a run performs, and every metric below aggregates over those experiments. A best score improves with the number of trials for reasons unrelated to the quality of the search, and a ratio computed over more proposals is estimated more precisely. Every metric is computed over the first three experiments, this being the number performed at the fixed-budget level and hence the number available in every run. Complete-run values are reported separately.

Outcome metrics. The gain over default is the difference between the score of the configuration finally recommended by the run and $a_0(d)$. It addresses the primary question concerning the result, namely whether the loop achieved any improvement. A negative value indicates that the run terminated on a configuration inferior to untuned defaults.

The improvement rate is the proportion of executed trials whose score is strictly higher than the best score observed earlier in the same run. A run of eight trials with one improvement is close to guessing, whereas a high rate is evidence that the interpret step is informing the next proposal.

Efficiency metrics. The wasted trial ratio is the number of proposals rejected before execution, whether malformed or invalid against the whitelist, together with the number that exactly duplicate an earlier proposal in the same run, divided by the total number of proposals attempted. Both failure modes consume a turn without yielding a measurement. A high ratio indicates that the loop is either failing to satisfy the response format or failing to consult its own transcript.

The cost to best is the number of executor calls made up to and including the call that produced the highest score of the run, taking the earliest such call on ties. Exploration after that point does not count. Since every executed trial consumes the same fixed budget, this is the computation spent to reach the result.

One qualification is required. The evaluation-criterion component at level 1 names cross-validated accuracy, which is the scale on which the gain over default is computed, so part of any effect that component shows is the treatment agreeing with the instrument. For the first dataset the gain on held-out validation accuracy is reported as a robustness check, and an effect is claimed for that component only where both scales agree. The second dataset allows no such check, for the reason given above.

Regression model and inference. Code each component as an indicator variable

$$X_i = \begin{cases} +1 & \text{if component } i \text{ is at level } 1, \\ -1 & \text{if at level } 0, \end{cases} \qquad i = 1, \ldots, 5, \tag{1}$$

and model the outcome of one run as

$$Y \;=\; \beta_0 + \sum_{i=1}^{5} \beta_i X_i + \sum_{j=2}^{D} \gamma_j D_j + \varepsilon, \tag{2}$$

Table 2: Main-effect estimates $2\hat{\beta}_i$ per metric from equation (2), with heteroskedasticity-consistent standard errors and Benjamini–Hochberg adjusted p-values. The $\hat{\gamma}$ column is the fitted dataset fixed effect (Wine relative to Breast Cancer). Stars mark adjusted $p < 0.05$ (*), < 0.01 (**) and < 0.001 (***).

Metric	M	B	S	O	E	$\hat{\gamma}$	R^2
Gain over default	+0.001	-0.001	+0.001**	+0.000	+0.000	-0.001***	0.029
Wasted trial ratio	-0.023***	-0.034***	+0.044***	-0.075***	+0.005	-0.003	0.159
Improvement rate	+0.008	-0.026*	+0.034**	+0.006	+0.018	-0.064***	0.044
Cost to best	+0.06	-0.02	+0.05	-0.01	-0.11	-0.20***	0.025

where D_j indicates that the run was made on dataset j, the first dataset serving as the reference level. The γ_j are dataset fixed effects: they absorb the inherent difficulty of each task, its headroom above $a_0(d)$ and its response to the whitelist, which would otherwise sit in the residual and inflate every standard error. Because the factorial is crossed with the task suite, each X_i is orthogonal to every D_j by construction, so the $\hat{\beta}_i$ are within-task effects of the architectural components and are not contaminated by which tasks happen to be easy. Here β_0 denotes the mean across configurations on the reference dataset, β_i the effect of component i, and ε run-to-run error. Each X_i assumes each value equally often and independently of the remainder, so that $\hat{\beta}_i$ does not shift when another component enters or leaves the model, and $2\beta_i$ is the average change in Y produced by switching component i. Since all thirty-two configurations are executed, the interaction model is obtained at no additional cost,

$$Y \;=\; \beta_0 + \sum_{i=1}^{5} \beta_i X_i + \sum_{1 \le i < j \le 5} \beta_{ij} X_i X_j + \sum_{j=2}^{D} \gamma_j D_j + \varepsilon, \tag{3}$$

which is how RQ2 is tested.

Estimation proceeds by ordinary least squares on the individual runs rather than on the cell means. Because the design is balanced the coefficients are identical under either choice, while the error term now reflects run-to-run scatter. Balance would likewise equalise the standard errors if the error variance were constant, which it is not: the stopping rule fixes the experiment count at one level and frees it at the other, so within-cell variance differs systematically between the two halves of the design. Heteroskedasticity-consistent standard errors are used throughout, and the wasted trial ratio and the improvement rate are fitted on their natural scale, both being proportions. Multiplicity is handled by Benjamini–Hochberg adjustment, with the family taken as the treatment coefficients of one model on one metric.

Results. Table 2 gives the main-effect estimates and Figure 2 plots them with their intervals. The components affect the four metrics unequally.

The wasted trial ratio is the most responsive metric. Four of the five components produce effects that survive adjustment, and the model accounts for $R^2 = 0.16$ of its variance. The reporting style produces the largest effect measured anywhere in the design, $2\hat{\beta}_O = -0.075$, corresponding to a reduction in the mean ratio from 0.106 at level 0 to 0.031 at level 1. Search breadth and the evaluation criterion act in the same direction, with effects of -0.034 and -0.023. The adaptive stopping rule acts in the opposite direction, $+0.044$, raising the mean ratio from 0.046 to 0.090. This is consistent with its mechanism: the adaptive level raises the mean number of executed experiments from 2.88 to 4.57, and a proportion of the additional proposals duplicate configurations already attempted.

The two outcome metrics, by contrast, respond to the stopping rule alone. It is the only component whose effect on the gain over default survives adjustment ($+0.001$, adj. $p = 0.005$), and it raises the improvement rate from 0.135 to 0.170. Search breadth is the only other component affecting the improvement rate, at -0.026. No component affects the cost to best at the adjusted 0.05 level; the largest estimate is exploit-first at -0.11, with adj. $p = 0.057$. The components that determine efficiency are therefore not the components that determine outcome.

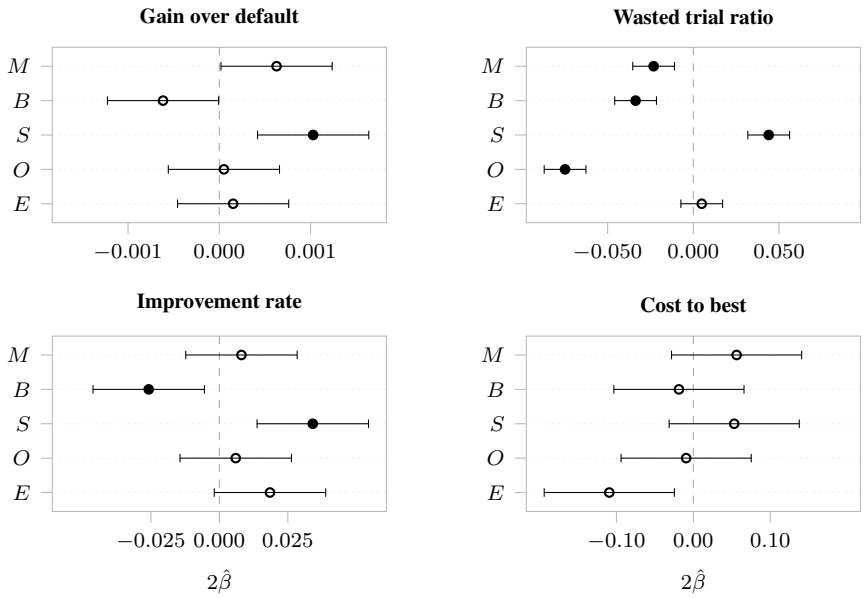

Figure 2: Estimated component effects on every metric. Points are $2\hat{\beta}_i$ from equation (2), bars are 95% intervals built from heteroskedasticity-consistent standard errors, and a filled point marks an effect that survives Benjamini–Hochberg adjustment at 0.05. The dataset fixed effect is in the model, so these are within-task effects. The four panels carry independent horizontal scales, the metrics being in different units.

Table 3: Two-way interaction estimates $2\hat{\beta}_{ij}$ from equation (3), and compliance per component. The four interactions with the smallest adjusted p are shown, with adjusted p given directly; compliance is reported as level 0 / level 1.

Interaction	Estimate	adj. p	Component	Compliance
$S \times O$ (wasted trial ratio)	-0.031	0.000	B	0.66 / 0.82
$B \times O$ (wasted trial ratio)	+0.018	0.009	S	0.89 / 0.20
$O \times E$ (wasted trial ratio)	-0.016	0.019	E	0.88 / 0.69
$M \times B$ (wasted trial ratio)	-0.010	0.171	O	205 / 386 chars
			M	not observable

Furthermore, on two of the four metrics the dataset fixed effect exceeds every component effect. Wine reduces the improvement rate by 0.064 and the cost to best by 0.20 relative to Breast Cancer, both at $p < 10^{-5}$, against largest component effects of 0.034 and 0.11 on the same metrics. This is the variation that equation (2) removes from the residual.

Table 3 gives the two-way interactions. Three of the forty estimated across the four metrics survive adjustment. All three occur on the wasted trial ratio and all three involve the reporting style: $S \times O$ at -0.031, $B \times O$ at $+0.018$, and $O \times E$ at -0.016. Of these, only $S \times O$ is comparable in magnitude to a main effect. Its sign indicates that the verbose reporting style and the adaptive budget are partial substitutes: the reduction in wasted trials attributable to either component is smaller when the other is already at level 1. For the remaining pairs the data provide no evidence of interaction, so the components may be treated as additive. This is the answer to RQ2, and the complete factorial is what permits the question to be tested rather than assumed.

An instruction is not invariably obeyed. Table 3 therefore also reports the proportion of runs whose behaviour matched the instruction given, so that a component without effect can be distinguished from one the model disregarded. Three components constrain an observable action and admit a mechanical check:

B by the number of model families the run proposes, S by the number of experiments it performs, E by whether its second proposal refines the first. O constrains wording rather than action, so it is assigned a manipulation check instead, namely the mean number of characters of hypothesis and interpretation text per run at each level. M leaves no behavioural trace at all, because the executor computes the score whatever the instruction says. The manipulation check separates the two levels of the reporting style clearly: the mean quantity of model-written text per run is 205 characters at level 0 and 386 at level 1. Compliance with the remaining instructions is, however, only partial. The fixed budget is followed in 89% of runs, whereas the adaptive rule is followed in only 20%. The stopping-rule effects reported above measure the effect of offering an adaptive budget, not the effect of a loop that terminates on diminishing returns. Search breadth is followed in 66% of narrow runs and 82% of broad runs, and exploit-first in 69% of runs. All estimates in Table 2 are consequently intention-to-treat estimates, attenuated towards zero in proportion to the non-compliance rate of the component concerned.

Taken together, the recovered effect structure is concentrated on process. Outcome is barely affected. Four of the five components move the wasted trial ratio, the largest being the reporting style at -0.075, which cuts the mean ratio from 0.106 to 0.031. The gain over default answers only to the stopping rule, and by $+0.001$ on the accuracy scale, a quantity smaller in absolute value than the -0.0013 contributed by the choice of dataset. Two components move the improvement rate, the stopping rule at $+0.034$ and search breadth at -0.026. On the cost to best nothing reaches the adjusted 0.05 level. Among the two-way interactions only $S \times O$ (-0.031) is comparable in magnitude to a main effect, and its sign indicates substitution, not reinforcement.

Two conclusions follow. First, within the range of wordings tested, the instruction file determines the efficiency of the search but not the quality of the configuration returned. Requiring the model to state its reasoning reduces the share of malformed and duplicate proposals by approximately two thirds, whereas no component produced a comparable change in the accuracy of the recommended configuration. Second, the outcome effects are bounded by the difficulty of the task. Default hyperparameters already achieve $a_0 = 0.960$ on both datasets, 40.0% of runs improve on that value and 20.9% terminate below it, so the available headroom is small. The measured effects concern the efficiency with which a ceiling is approached, and say nothing about the height of that ceiling. Whether the same structure holds on tasks with greater headroom is not determined by this design.

Four boundary conditions delimit these results. Every finding is conditional on one agent model, so what has been measured is the action of instructions on a single policy, not a property of instructions in general. Each level is realised by one generated paragraph: the semantics of a component were held fixed, but its phrasing was not varied, so the variance attributable to wording cannot be separated from the variance attributable to the architectural constraint itself. The evaluation covers two datasets, which leaves the dataset fixed effect resting on a single contrast. The search space is a small discrete set of hyperparameters rather than code. Within those bounds the study is a proof of concept, and its claim is correspondingly narrow: that editing the constraints in `program.md` has a measurable causal influence on the loop's behaviour. Establishing the size and the generality of that influence is left to the wider designs these conditions leave open.

4 Conclusion

In an autoresearch loop, `program.md` performs the function that controller code performs in conventional software, yet it cannot be read in the manner of code. It is treated here as a configuration vector. Five components of `program.md` were written at two levels each, all $2^5 = 32$ configurations were generated and executed against a real executor on two datasets with twenty replicates apiece, and the resulting behaviour was reduced to four outcome metrics and regressed on an orthogonal ± 1 coding of the components. Because the configurations were constructed rather than collected, the coefficients admit a causal reading. Because the factorial is complete, the independence of the components could be tested.

The components determined the process, not the outcome. The reporting style produced the largest effect in the design, reducing the mean wasted trial ratio from 0.106 to 0.031, and search breadth and the evaluation criterion moved the same metric by -0.034 and -0.023. The stopping rule was the only component with an effect on either outcome metric, $+0.001$ on the gain over default and $+0.034$ on the improvement rate, and no component determined the cost to best. Three of forty two-way interactions survived adjustment, all on the wasted trial ratio and all involving the reporting style, so the components may be treated as additive. On two of the four metrics the dataset fixed effect exceeded every component effect, which is the reason equation (2) controls for it. For tasks of this difficulty the instruction file determines how the loop conducts its search rather than what the search returns.

The method is not tied to the five components varied here. Any instruction file that can be partitioned into slots may be studied in the same manner, and full enumeration remains inexpensive while the number of slots is small. Two extensions are indicated. Sampling several wordings per level would separate a component from its phrasing, which is the most serious of the limitations noted above. Applying the procedure to a loop that edits code rather than hyperparameters would test it in a setting where the instruction file carries considerably more of the system's behaviour.

References

[1] T. Lütke. Performance: 53% faster parse+render, 61% fewer allocations. Pull request #2056, Shopify/liquid, 2026. Produced by an autoresearch run of approximately 120 experiments over 93 commits; unmerged at the time of writing. `https://github.com/Shopify/liquid/pull/2056`.

[2] B. Tuychiev. A guide to Andrej Karpathy's AutoResearch: automating ML with AI agents. DataCamp tutorial, 23 March 2026. Secondary source; it reports the contents of the repository's `program.md` and results log. `https://www.datacamp.com/tutorial/guide-to-autoresearch`.

[3] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. Nature, 624(7992):570–578, 2023. doi:10.1038/s41586-023-06792-0.

[4] Q. Huang, J. Vora, P. Liang, and J. Leskovec. MLAgentBench: Evaluating language agents on machine learning experimentation. In Proceedings of the 41st International Conference on Machine Learning (ICML), PMLR 235:20271–20309, 2024. arXiv:2310.03302.

[5] A. Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. GitHub repository, 2026. `https://github.com/karpathy/autoresearch`.

[6] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery, 2024. arXiv:2408.06292.

[7] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr. Quantifying language models' sensitivity to spurious features in prompt design, or: How I learned to start worrying about prompt formatting. ICLR, 2024. arXiv:2310.11324.

[8] A. Webson and E. Pavlick. Do prompt-based models really understand the meaning of their prompts? NAACL-HLT, pages 2300–2344, 2022.

A The files the loop runs on

Section 1 describes the three-file contract of Karpathy's `autoresearch`: `prepare.py` fixes the data and the score, `train.py` holds the model and the training loop, and `program.md` holds the instructions. The study keeps that contract file for file. All three are reproduced below: the specification of `program.md` in Appendix B, `prepare.py` in Appendix C, and `train.py` in Appendix D.

One thing does differ, and it is the narrowing of Section 2 rather than a departure from the contract. In Karpathy's loop the agent edits `train.py` directly. Here the search space is a hyperparameter space, not arbitrary code, so that the instruction file stays the only free variable; the agent names a model family and hyperparameter values in a JSON proposal and `train.py` builds exactly that estimator. The agent therefore acts on `train.py` through a fixed interface instead of rewriting it. What is unchanged is the ownership that matters for measurement: `prepare.py` is reachable by nobody, so no level of any component can alter what a score means, and every number in a transcript is a measurement rather than a claim.

B Specification of the instruction file

A `program.md` is not written; it is rendered. This appendix gives the whole specification, from which any of the thirty-two files can be reconstructed exactly: the template, which is fixed, and the ten paragraphs that fill its five slots.

Appendix B.1 gives the template. Everything in it is byte-identical across all thirty-two configurations of a task. The five marked slots are the independent variables. The three remaining substitutions, `task_title`, `research_question` and `dataset_name`, are set by the task and not by the configuration, so they are constant across the thirty-two files of one dataset and differ only between datasets.

Appendix B.2 gives the ten paragraphs, two per slot, verbatim. A configuration $c = (c_1, \ldots, c_5)$ selects one paragraph per slot, and substituting one paragraph for the other level of the same slot is the only difference between two files. These ten texts are the treatment: they were written once by a language model from fixed meta-prompts, then frozen and reused word for word, for the reason given in Section 2. Table 1 summarises them; what follows is the text itself, since it is the wording rather than the summary that the loop responds to.

B.1 Template

```
# Research Task: {{ task_title }}

You are an autonomous research agent. Your task is to investigate the following
question through real, executed experiments (not simulated ones):

> {{ research_question }}

You have access to a fixed dataset ({{ dataset_name }}, already loaded and split
into train/validation) and to a tool that fits either a RandomForestClassifier or
a GradientBoostingClassifier with hyperparameters you choose. The tool always
returns the same record: 5-fold cross-validated accuracy on the training split,
its standard deviation across folds, and accuracy on the held-out validation
split.

## Evaluation

⟨ slot M --- metric ⟩
{{ metric_text }}

## Search strategy

⟨ slot B --- breadth ⟩
{{ breadth_text }}

⟨ slot E --- emphasis ⟩
{{ emphasis_text }}

## Budget and stopping

⟨ slot S --- stopping ⟩
{{ stopping_text }}

## Reporting style

⟨ slot O --- output format ⟩
{{ output_format_text }}
```

```
## Required response format

At every step, respond with a single JSON object and nothing else, matching one
of these two shapes:

Proposing an experiment:
```json
{"action": "propose", "hypothesis": "<why you think this configuration will help>",
 "model": "random_forest" | "gradient_boosting",
 "params": {"<hyperparameter>": <value>, ...}}
```

Interpreting the last result and deciding what happens next:
```json
{"action": "interpret", "interpretation": "<what the result tells you>",
 "decision": "continue" | "stop",
 "final_recommendation": "<only required when decision == stop: your final
   recommended model and hyperparameters, and why>"}
```
```

B.2 The ten treatment paragraphs

slot M --- Evaluation criterion (substituted for {{ metric_text }})
level 0: Evaluate model quality in whatever way you judge appropriate.

level 1: Evaluate every model using 5-fold cross-validated accuracy on the training split, AND report the standard deviation across folds as a measure of stability. A configuration is only better than another if its mean accuracy is higher; use the standard deviation only to break ties or to flag instability.

slot B --- Search breadth (substituted for {{ breadth_text }})
level 0: Stay within a single model family per experiment. Pick either RandomForestClassifier or GradientBoostingClassifier for the whole session unless you have exhausted reasonable hyperparameters for it and explicitly decide to switch.

level 1: You are free to compare RandomForestClassifier and GradientBoostingClassifier side by side within the same session, and to vary multiple hyperparameters of each at once.

slot S --- Stopping rule (substituted for {{ stopping_text }})
level 0: You have a fixed budget of exactly 3 experiments in total. Run exactly 3, then stop and report your final recommendation, regardless of the outcome of experiment 3.

level 1: You have a budget of up to 8 experiments. After each experiment, decide whether to continue: stop as soon as an experiment fails to improve the evaluation metric (defined above) by more than 0.002 over the best result so far, or when you reach 8 experiments, whichever comes first.

slot O --- Reporting style (substituted for {{ output_format_text }})
level 0: Keep every response short: state the action you are taking and the numbers involved, with minimal prose.

level 1: Explain your reasoning at every step in full sentences: what you hypothesize, why, what you expect to observe, and how you interpret the actual result relative to that expectation.

slot E --- Search emphasis (substituted for {{ emphasis_text }})
level 0: Prioritize breadth of search: try several meaningfully different configurations before you spend more than one experiment refining any single one of them.

level 1: Prioritize depth: as soon as one configuration looks promising, immediately refine it with nearby variations (e.g. slightly different hyperparameter values) before trying anything unrelated.

C `prepare.py`

The data and the score. Nobody edits this file, and the agent never sees it, so no level of any component can change what an experiment is measured on or how it is scored. One stratified $70/30$ partition per task, fixed with the same seed for every run, and one scoring procedure returning the same record every time, whatever a configuration's evaluation paragraph emphasises.

```
"""
prepare.py -- the data and the score. Nobody edits this file.

The first of the three files of the autoresearch contract (thesis, Introduction).
It fixes what every experiment is measured on and how it is scored, so that a
```

```
score is comparable across configurations, across replicates and across the whole
study. Neither the agent nor the instruction file can reach it: the agent never
sees this module, and no level of any component in program.md changes what it
returns. That is what makes a number in a transcript a measurement rather than a
claim.

Two responsibilities, and no others:

  load(dataset)              the fixed train/validation partition for one task
  score(estimator, dataset)  the record every experiment is scored by

Every task uses the identical split protocol: one stratified 70/30 partition
fixed with the same seed across all runs, so the split is a constant of the study
rather than a source of variation (thesis, Experimental setup). The score is the
same record whatever a configuration's evaluation paragraph emphasises:
cross-validated accuracy on the training split, its across-fold standard
deviation, and accuracy on the held-out validation split.

Adding a dataset here does not put it in the study; the suite is whatever
study_config.DATASETS says.
"""
import sys
from functools import lru_cache
from pathlib import Path

from sklearn import datasets as skds
from sklearn.model_selection import cross_val_score, train_test_split

sys.path.insert(0, str(Path(__file__).resolve().parent / "config"))
from study_config import CV_FOLDS, SCORING, SPLIT_SEED, TEST_SIZE  # noqa: E402

# name -> zero-argument loader returning (X, y)
LOADERS = {
    "breast_cancer": lambda: (lambda d: (d.data, d.target))(skds.load_breast_cancer()),
    "wine":          lambda: (lambda d: (d.data, d.target))(skds.load_wine()),
    "digits":        lambda: (lambda d: (d.data, d.target))(skds.load_digits()),
    "iris":          lambda: (lambda d: (d.data, d.target))(skds.load_iris()),
}


@lru_cache(maxsize=None)
def load(name):
    """Return the fixed split for one dataset as (X_train, X_val, y_train, y_val)."""
    if name not in LOADERS:
        raise KeyError(f"unknown dataset {name!r}; known: {sorted(LOADERS)}")
    X, y = LOADERS[name]()
    return train_test_split(
        X, y, test_size=TEST_SIZE, random_state=SPLIT_SEED, stratify=y
    )


def score(estimator, dataset):
    """The one scoring procedure of the study. Returns the record, or raises."""
    X_tr, X_va, y_tr, y_va = load(dataset)
    cv = cross_val_score(estimator, X_tr, y_tr, cv=CV_FOLDS, scoring=SCORING)
    estimator.fit(X_tr, y_tr)
    val = estimator.score(X_va, y_va)
    return {
        "cv_accuracy_mean": round(float(cv.mean()), 6),
        "cv_accuracy_std": round(float(cv.std()), 6),
        "val_accuracy": round(float(val), 6),
    }


def describe(name):
    import numpy as np
    X_tr, X_va, y_tr, y_va = load(name)
    return {
        "dataset": name,
        "n_train": int(X_tr.shape[0]),
        "n_val": int(X_va.shape[0]),
        "n_features": int(X_tr.shape[1]),
        "n_classes": int(len(np.unique(y_tr))),
    }
```

D `train.py`

The model and the training loop. The whitelist here is the boundary of the search space: a proposal naming an unknown family, or a hyperparameter outside the whitelist for its family, is rejected before it reaches the scoring procedure, so a malformed proposal costs a turn but not an experiment.

```
"""
train.py -- the model and the training loop.

The second file of the autoresearch contract (thesis, Introduction). In
Karpathy's loop this is the file the agent edits, and it is the only one it may
edit. The loop studied here narrows the search space from arbitrary code to a
hyperparameter space, so that the instruction file stays the only free variable
(thesis, The loop under study). The narrowing changes how the agent reaches this
file, not which file it acts on: instead of rewriting the source, the agent names
a model family and hyperparameter values in a JSON proposal, and this module
builds and trains exactly that estimator. Everything the agent can vary about the
model lives here; nothing it can vary about the measurement does, that being
prepare.py.

The whitelist is the boundary of the search space. A proposal naming an unknown
family, or a hyperparameter outside the whitelist for its family, is rejected
here and never reaches the scoring procedure, which is what keeps a malformed
proposal from costing an experiment.

    run_experiment(model, params, seed, dataset) -> record | {"error": ...}
"""
import sys
import time
from pathlib import Path

from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier

STUDY = Path(__file__).resolve().parent
sys.path.insert(0, str(STUDY))
sys.path.insert(0, str(STUDY / "config"))
import prepare  # noqa: E402
import study_config  # noqa: E402  (read live, so WORKERS can set EXECUTOR_N_JOBS)
from study_config import ALLOWED_PARAMS  # noqa: E402


def build(model, params, seed):
    """The estimator named by a proposal. Raises on an unknown family."""
    if model == "random_forest":
        # n_jobs is set to 1 when the outer loop is parallelised, so that the
        # forest's internal threads do not oversubscribe the cores.
        return RandomForestClassifier(
            random_state=seed, n_jobs=study_config.EXECUTOR_N_JOBS, **params)
    if model == "gradient_boosting":
        return GradientBoostingClassifier(random_state=seed, **params)
    raise ValueError(f"unknown model {model!r}")


def validate(model, params):
    """Whitelist check. Returns an error string, or None if the proposal is legal."""
    if model not in ALLOWED_PARAMS:
        return f"unknown model '{model}', must be one of {sorted(ALLOWED_PARAMS)}"
    bad = set(params) - ALLOWED_PARAMS[model]
    if bad:
        return f"unsupported params for {model}: {sorted(bad)}"
    return None


def run_experiment(model, params, seed, dataset):
    """Train one estimator on one dataset and score it. Record, or {'error': ...}."""
    err = validate(model, params)
    if err:
        return {"error": err}

    t0 = time.time()
    try:
        record = prepare.score(build(model, params, seed), dataset)
    except Exception as exc:                        # noqa: BLE001 - reported to the model
        return {"error": f"experiment failed: {exc}"}

    return {"model": model, "params": params, "dataset": dataset, **record,
            "wall_time_s": round(time.time() - t0, 3)}
```